

Sorbonne Université

Master Automatique, Robotique : Systèmes intelligents
UE UM4RBI22

Rapport Technique

Générateur de Donjon & Labyrinthe Interactif

Auteur : BOUANZOUL Ahmed Walid ; DEBZ Wail
Année : 2025–2026

Table des matières

1	Architecture Globale	2
1.1	Diagramme de Classes UML	2
1.2	Arborescence du projet	2
2	Description des Classes Principales	2
2.1	Case — Classe abstraite de base	2
2.2	Sous-classes concrètes de Case	3
2.3	CaseFactory — Design Pattern Factory	4
2.4	Donjon — Grille et algorithmes	4
2.5	Aventurier — Le joueur	4
3	Algorithmes Utilisés	5
3.1	Recursive Backtracking (Génération du labyrinthe)	5
3.2	BFS — Breadth-First Search (Pathfinding)	6
3.3	Placement aléatoire des éléments	7
4	Structures de Données — Justification	7
5	Concepts C++ Mis en Œuvre	8
6	Difficultés Rencontrées et Solutions	8
6.1	Dépendance circulaire Case $\leftrightarrow$ Aventurier	8
6.2	Dimensions paires cassent l'algorithme	8
6.3	Fuites mémoire (raw pointers)	8
6.4	Scintillement de l'affichage console	8
6.5	Mécanisme de fuite face aux monstres	9
6.6	Garantie de chemin entrée $\rightarrow$ sortie	9

1 Architecture Globale

1.1 Diagramme de Classes UML

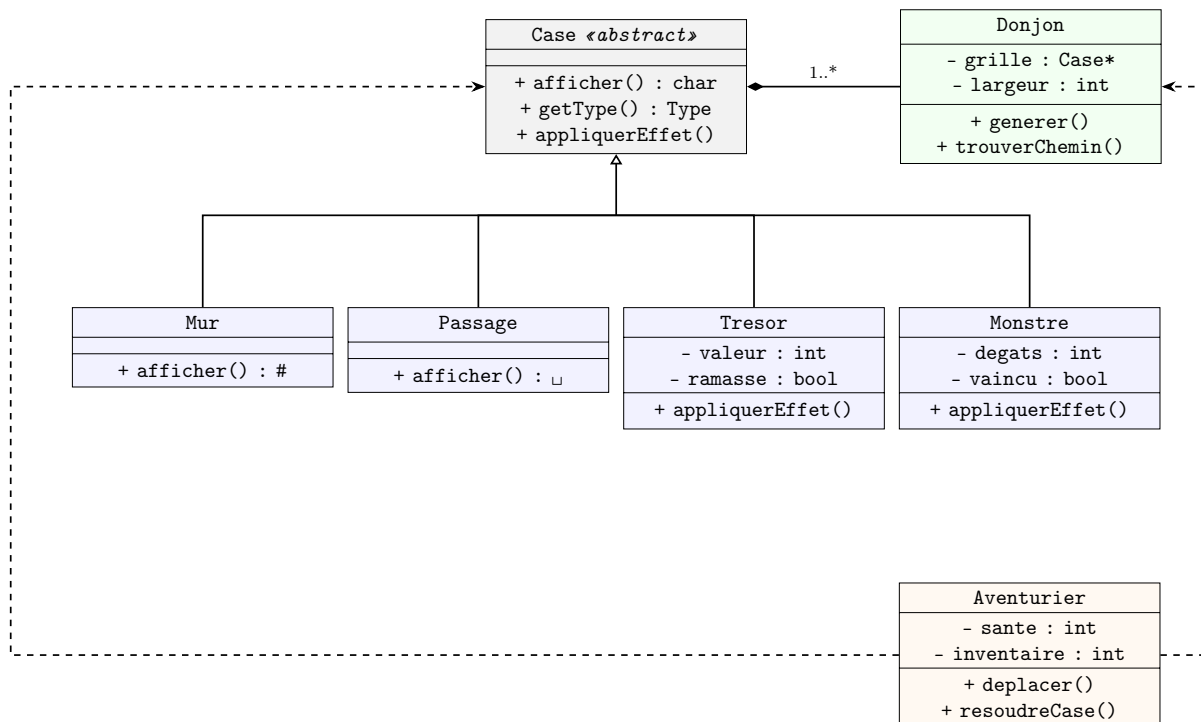


FIGURE 1 – Architecture logicielle - Hiérarchie des classes et interactions

1.2 Arborescence du projet

```
labyrinthe-main/
main.cpp           # Point d'entrée
Makefile           # Build (g++ C++17)
Exemple de donjons.txt # Donjons sauvegardés
src/
  constants.h      # Constantes globales
  TypeCase.h       # Enum TypeCase
  classes/
    Case.h         # Classe abstraite de base
    Mur.h / mur.cpp
    Passage.h / passage.cpp
    Tresor.h / tresor.cpp
    Monstre.h / monstre.cpp
    Pieu.h / piege.cpp
    CaseFactory.h / CaseFactory.cpp
    Donjon.h / donjon.cpp
    Aventurier.h / player.cpp
```

2 Description des Classes Principales

2.1 Case — Classe abstraite de base

Fichier : `Case.h`

Case est la **racine de la hiérarchie polymorphe**. Elle déclare quatre méthodes virtuelles pures :

Méthode	Signature	Rôle
<code>afficher()</code>	<code>virtual char afficher() const = 0</code>	Retourne le caractère ASCII de la case
<code>getType()</code>	<code>virtual TypeCase getType() const = 0</code>	Retourne le type énuméré
<code>estFranchissable()</code>	<code>virtual bool estFranchissable() const = 0</code>	Indique si le joueur peut marcher dessus
<code>appliquerEffet()</code>	<code>virtual void appliquerEffet(Aventurier&) = 0</code>	Applique l'effet sur le joueur

Elle inclut aussi un **destructeur virtuel** (`virtual ~Case() = default`) pour garantir la destruction correcte via pointeur de base, et une **surcharge de `operator<<`** (friend) pour l'affichage via `std::ostream`.

Important

La forward declaration `class Aventurier;` évite une dépendance circulaire entre `Case.h` et `Aventurier.h`.

2.2 Sous-classes concrètes de Case

Mur

- **Affichage** : # — **Franchissable** : false
- **Effet** : Aucun (corps vide). Le mur bloque le déplacement en amont via `estFranchissable()`.

Passage

- **Affichage** : (espace) — **Franchissable** : true
- **Effet** : Aucun. Case neutre.

Tresor

- **Attributs** : `valeur` (= 1 par défaut), `ramasse` (booléen d'état)
- **Affichage** : M si vivant, si vaincu
- **Effet** : Ajoute `valeur` à l'inventaire du joueur, passe `ramasse = true`, message console.

Monstre

- **Attributs** : `nom` ("Goblin"), `degats` (15 HP), `vaincu` (booléen)
- **Affichage** : M si vivant, si vaincu
- **Effet** : Propose un choix **Combattre (C)** / **Fuir (F)** :
 - **Combat** : inflige `degats` au joueur, passe `vaincu = true`
 - **Fuite** : repositionne le joueur à sa position précédente (`prevX`, `prevY`)

Piege

- **Attributs** : `degats` (20 HP), `active` (booléen)
- **Affichage** : T si actif, sinon
- **Effet** : Inflige `degats` automatiquement, se désactive (`active = false`). Usage unique.

2.3 CaseFactory — Design Pattern Factory

Méthode statique unique : `Case* creerCase(TypeCase type)`

Implémentation via un `switch` sur l'enum `TypeCase`. Chaque cas retourne un `new` de la sous-classe correspondante. Le cas `default` retourne un `Mur` par sécurité.

Avantages dans le projet :

1. **Découplage** : Donjon ne connaît jamais les classes concrètes, uniquement `Case*` et `TypeCase`
2. **Extensibilité** : Ajouter un type (ex : `Porte`) = modifier l'enum + ajouter un `case` dans la factory
3. **Lisibilité** : `creerCase(TypeCase::TRESOR)` est plus expressif qu'un `new Tresor()` dispersé

2.4 Donjon — Grille et algorithmes

Attributs :

- `grille` : `vector<vector<Case*>>` — matrice 2D de pointeurs polymorphes
- `largeur`, `hauteur` : dimensions (forcées impaires pour l'algorithme)
- `entree` : `pair<int,int> = (1,1)`
- `sortie` : `pair<int,int> = (largeur-2, hauteur-2)`

Méthodes clés :

Méthode	Description
<code>generer(larg, haut)</code>	Initialise la grille en murs, lance le backtracking, place entrée/sortie/éléments
<code>genererLabyrinthe(x, y)</code>	Recursive Backtracking — creuse les passages
<code>placerElements()</code>	Place trésors (5%), monstres (5%), pièges (3%) aléatoirement
<code>trouverChemin(dep, arr)</code>	BFS — retourne le chemin le plus court
<code>afficher(joueur)</code>	Affichage ASCII avec joueur @ et sortie E
<code>afficherChemin(joueur, chemin)</code>	Affichage avec le chemin marqué .

Destructeur : Parcourt toute la grille et `delete` chaque `Case*` pour éviter les fuites mémoire.

2.5 Aventurier — Le joueur

Attributs :

- `posX`, `posY` : position courante
- `prevX`, `prevY` : position précédente (utilisée pour la fuite face aux monstres)
- `sante` : points de vie (initialisés à 100)
- `inventaire` : compteur de trésors collectés

Méthodes clés :

Boucle de jeu : `while (estVivant() && position != sortie)` — à chaque itération :

1. Clear screen (cross-platform : `cls` Windows, `clear` Linux)
2. Affiche le donjon avec le joueur
3. Lit l'input clavier (z/q/s/d/p)

Méthode	Description
<code>deplacer(nx, ny)</code>	Sauvegarde la position précédente, met à jour la courante
<code>resoudreCase(Case* c)</code>	Délègue à <code>c->appliquerEffet(*this)</code> — polymorphisme pur
<code>estVivant()</code>	<code>return sante > 0</code>
<code>boucleDeJeu(Donjon& d)</code>	Boucle principale : affichage → input → vérification → déplacement → effet
<code>operator«</code>	Surcharge friend pour affichage formaté

4. Vérifie les bornes et la franchissabilité
5. Déplace et résout l'effet de la case

3 Algorithmes Utilisés

3.1 Recursive Backtracking (Génération du labyrinthe)

Principe

L'algorithme génère un **labyrinthe parfait** (exactement un chemin entre deux cases quelconques) par parcours en profondeur randomisé avec retour arrière.

Pseudo-code implémenté

```

genererLabyrinthe(x, y):
    marquer grille[y][x] comme PASSAGE
    directions = [{0,-2}, {0,+2}, {-2,0}, {+2,0}]
    melanger(directions) // aleatoire via mt19937

    pour chaque direction (dx, dy):
        nx = x + dx, ny = y + dy
        si (nx, ny) dans bornes ET grille[ny][nx] == MUR:
            casser le mur intermediaire : grille[(y+ny)/2][(x+nx)/2] = PASSAGE
            genererLabyrinthe(nx, ny) // appel recursif

```

Justification du choix

Critère	Recursive Backtracking	Alternatives (Kruskal, Prim)
Complexité	$O(n)$ — visite chaque cellule une fois	$O(n \log n)$ pour Kruskal
Simplicité d'implémentation	~20 lignes de code récursif	Nécessite Union-Find ou tas
Qualité du résultat	Longs corridors sinueux, très immersif	Prim : branches courtes, moins de "profondeur"

Pourquoi les dimensions sont impaires

La grille utilise une convention où les **cellules exploitables** sont aux coordonnées impaires (1,1), (1,3), (3,1)... et les **murs intermédiaires** aux coordonnées paires. Forcer `largeur` et `hauteur` impairs (`larg % 2 == 0 ? larg+1 : larg`) garantit que la bordure extérieure reste un mur solide et que chaque cellule a bien un mur séparateur vers ses voisins.

Structures de données

- `int directions[4][2]` : Tableau statique des 4 directions (pas de 2 cases). Mélangé via `std::shuffle + std::mt19937` pour le caractère aléatoire.
- `std::mt19937 rng` : Générateur Mersenne Twister initialisé par `std::random_device`. Choisi pour sa qualité statistique supérieure à `rand()`.

3.2 BFS — Breadth-First Search (Pathfinding)

Principe

BFS explore le graphe **niveau par niveau** depuis le départ. Sur une grille non pondérée, il garantit le **chemin le plus court** en nombre de cases.

Implémentation

```
trouverChemin(depart, arrivee):
    file = queue vide
    visite = tableau 2D de bool (false)
    parent = tableau 2D de pair<int,int> ((-1,-1))

    enfiler(depart)
    visite[depart] = true

    tant que file non vide:
        courant = defiler()
        si courant == arrivee:
            reconstruire et retourner le chemin via parent[]

        pour chaque voisin (4 directions, pas de 1):
            si dans bornes ET non visite ET franchissable:
                visite[voisin] = true
                parent[voisin] = courant
                enfiler(voisin)

    retourner [] // aucun chemin
```

Reconstruction du chemin

On remonte de `arrivee` vers `depart` via le tableau `parent`, puis on inverse le vecteur avec `std::reverse`.

Justification du choix

Critère	BFS	DFS	Dijkstra	A*
Optimalité (grille non pondérée)	Garanti	Non	Oui	Oui
Complexité	$O(V + E)$	$O(V + E)$	$O(V \log V)$	$O(V \log V)$
Implémentation	Simple (queue)	Simple (pile)	Priority queue	Priority q. + heuristique
Adapté au problème	Parfait	Pas optimal	Surdimensionné	Surdimensionné

BFS est le choix **optimal** ici : la grille est non pondérée (chaque case = 1 pas), donc BFS trouve le plus court chemin avec la complexité minimale.

Structures de données utilisées

Structure	Type C++	Justification
File (FIFO)	<code>std::queue<pair<int,int>></code>	$O(1)$ pour push/pop, sémantique FIFO requise par BFS
Tableau de visite	<code>vector<vector<bool>></code>	Accès $O(1)$ par coordonnées, évite les revisites
Tableau de parents	<code>vector<vector<pair<int,int>>></code>	Permet la reconstruction du chemin en $O(\text{longueur})$
Ensemble du chemin	<code>std::set<pair<int,int>></code>	Utilisé dans <code>afficherChemin()</code> pour un lookup $O(\log n)$ rapide

3.3 Placement aléatoire des éléments

Distribution par **seuils cumulatifs** sur un tirage uniforme $[0, 99]$:

```

r $in$ [0, 4] $> Tresor (5%)
r $in$ [5, 9] $> Monstre (5%)
r $in$ [10, 12] $> Piege (3%)
r $in$ [13, 99] $> Inchange (87%)

```

Les cases d'entrée et de sortie sont **exclues** du placement pour garantir un départ et une arrivée sûrs.

4 Structures de Données — Justification

Structure	Utilisation	Justification
<code>vector<vector<Case*>></code>	Grille du donjon	Matrice 2D dynamique, accès $O(1)$ par indices, taille paramétrable
<code>pair<int,int></code>	Positions (entrée, sortie, joueur)	Léger, sémantique (x,y), compatible STL
<code>enum class TypeCase</code>	Types de cases	Type-safe, pas de collision de noms, switch exhaustif
<code>std::queue</code>	BFS	FIFO natif, $O(1)$ amortie
<code>std::set</code>	Lookup chemin pour affichage	$O(\log n)$ pour <code>count()</code> , évite parcours linéaire
<code>std::mt19937</code>	Génération aléatoire	Qualité statistique, reproductible avec seed
<code>std::ostringstream</code>	Buffer d'affichage	Évite le scintillement en construisant tout l'affichage avant un seul <code>cout</code>

5 Concepts C++ Mis en Œuvre

Concept	Où dans le code
Héritage polymorphe	Case → Mur, Passage, Tresor, Monstre, Pieu
Méthodes virtuelles pures	afficher(), getType(), estFranchissable(), appliquerEffet()
Destructeur virtuel	virtual ~Case() = default
Design Pattern Factory	CaseFactory::creerCase(TypeCase)
Surcharge d'opérateurs	operator« pour Case et Aventurier
Forward declarations	class Aventurier; dans Case.h et Donjon.h
Enum class (scoped)	TypeCase avec 5 valeurs
Compilation conditionnelle	#ifdef _WIN32 pour clear screen cross-platform
RAII / Gestion mémoire	Destructeur de Donjon libère toute la grille
Constantes constexpr	constants.h — évaluation à la compilation

6 Difficultés Rencontrées et Solutions

6.1 Dépendance circulaire Case ↔ Aventurier

Problème : `Case::appliquerEffet()` prend un `Aventurier&`, mais `Aventurier` utilise des `Case*`. Inclusion croisée → erreur de compilation.

Solution : **Forward declaration** `class Aventurier;` dans `Case.h`. L'inclusion complète de `Aventurier.h` se fait uniquement dans les `.cpp` des sous-classes.

6.2 Dimensions paires cassent l'algorithme

Problème : Le Recursive Backtracking saute de 2 en 2 (cellules impaires). Avec des dimensions paires, la dernière colonne/ligne est inaccessible et la bordure est incomplète.

Solution : Forcer les dimensions impaires dans `generer()` :

```
largeur = (larg % 2 == 0) ? larg + 1 : larg;
hauteur = (haut % 2 == 0) ? haut + 1 : haut;
```

6.3 Fuites mémoire (raw pointers)

Problème : La grille utilise des `Case*` (raw pointers) alloués via `new`. Sans gestion explicite, chaque remplacement de case fuit.

Solution :

- `delete` systématique avant chaque remplacement dans `genererLabyrinthe()`, `placerElements()` et `setCase()`
- Destructeur `~Donjon()` parcourt et libère toute la grille
- Trade-off : `unique_ptr` aurait été plus sûr mais le sujet imposait des `Case*`

6.4 Scintillement de l'affichage console

Problème : Afficher case par case avec `cout` provoque un scintillement visible à chaque rafraîchissement.

Solution : Construire tout l'affichage dans un `std::ostringstream` buffer, puis faire un seul `cout` « `buffer.str()` atomique.

6.5 Mécanisme de fuite face aux monstres

Problème : Quand le joueur fuit un monstre, il doit revenir à sa position précédente. Mais `deplacer()` a déjà écrasé la position.

Solution : Attributs `prevX` / `prevY` dans `Aventurier`, mis à jour **avant** chaque déplacement. La fuite appelle `joueur.deplacer(prev.first, prev.second)`.

6.6 Garantie de chemin entrée → sortie

Problème : Le Recursive Backtracking produit un labyrinthe parfait, mais les cases d'entrée/sortie pourraient rester des murs si mal positionnées.

Solution : Après la génération, vérification explicite :

```
if ( grille[entree.second][entree.first] -> getType() != TypeCase::PASSAGE) {  
    delete grille[entree.second][entree.first];  
    grille[entree.second][entree.first] = CaseFactory::creerCase(TypeCase::PASSAGE);  
}
```

Idem pour la sortie. Le labyrinthe parfait garantit ensuite qu'un chemin unique existe.